

INTELLIGENCE BASED GENERATIVE MODEL FOR PROGRAMMING LANGUAGE

Authors: Agatheeswaran A, Hasseb Mohammed H, Praveenkumar S.
Mr R.Gobinath (ME.PhD)-Assistant Professor

AI&DS Dep-EGS Pillay Engineering College

1. INTRODUCTION

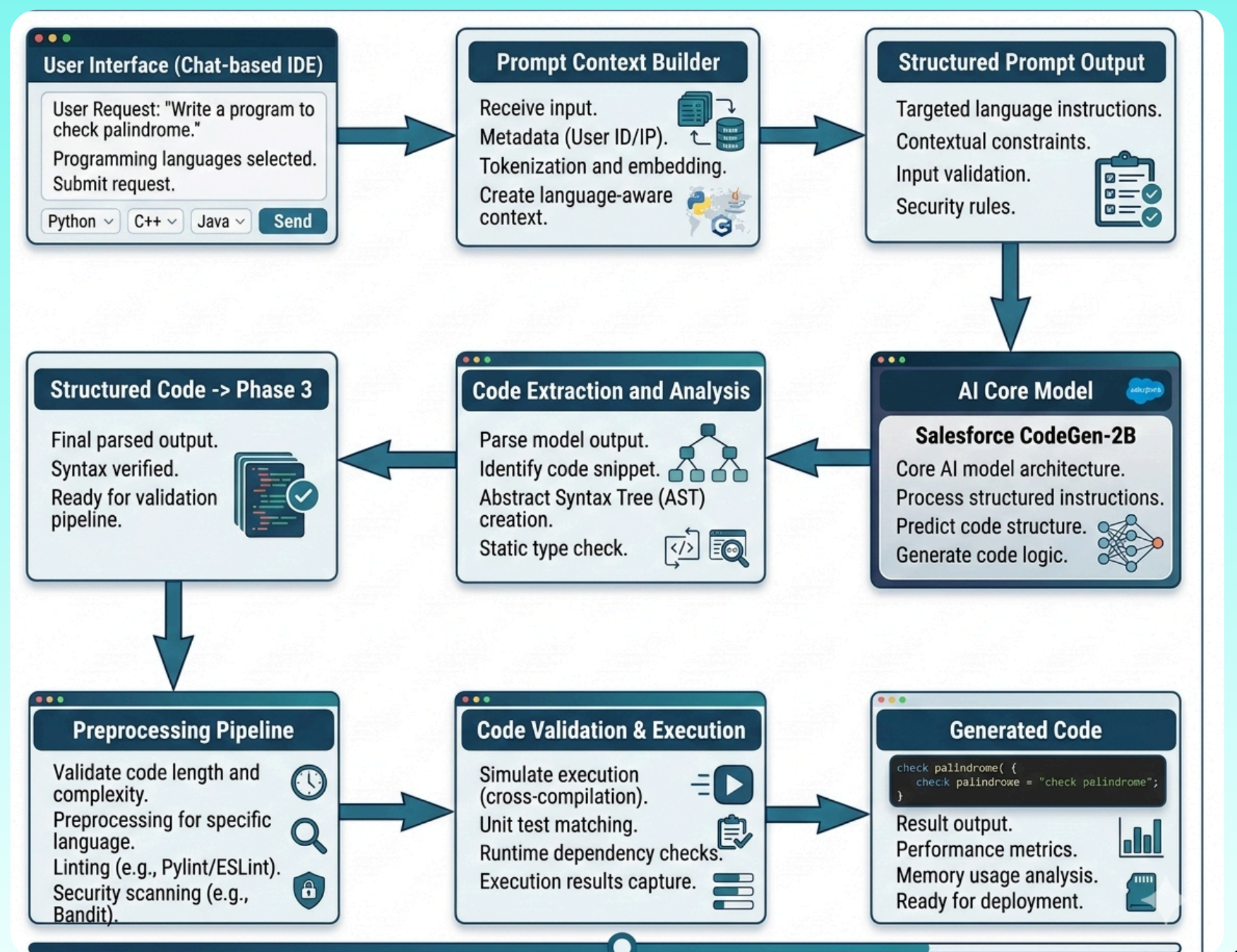
A unified AI ecosystem designed to automate code generation and debugging. Built on the **CodeGen-2B-Multi** architecture, this generative model understands syntax and semantics across various languages, transforming natural language instructions into functional, optimized source code to accelerate the **SDLC**.



2. METHOD:

The system architecture utilizes a multi-tiered approach, centering on the **CodeGen-2B-Multi** model, a 2-billion parameter Large Language Model optimized for multi-step program synthesis. The backend is using the **FastAPI** framework, employing asynchronous processing and structured prompt templates to ensure high-speed, code generation. For the user interface, a responsive frontend was developed using **Vanilla JavaScript** and **CSS3**, providing real-time environment for code debugging, translation, and generation across **Python, Java, and C++**.

3. ARCHITECTURE



3.1. DATA FORMAT

```
{
  "prompt": "Write a Python function to add two numbers.",
  "completion": "def add(a, b): return a + b"
},
{
  "prompt": "Create a JavaScript function to check if a number is even.",
  "completion": "const isEven = (num) => num % 2 === 0;"
},
{
  "prompt": "Write a C program to print 'Hello, World!'.",
  "completion": "#include <stdio.h>\nint main() { printf(\"Hello, World!\\n\"); return 0; }"
}
```

4. Methodology

Model Selection: Integrating CodeGen-2B-Multi, a 2-billion parameter LLM optimized for multi-step program synthesis.

Backend Engineering: A high-performance, asynchronous FastAPI efficient model inference.

Prompt Engineering: Structured instruction-following templates (### Instruction: / Code:) for high-accuracy code generation.

Frontend Design: A lightweight Vanilla JavaScript & CSS3 responsive code editor.

Validation: Tested via 'Zero-Shot' learning tasks code debugging, cross-language translation, and semantic explanation.

5. Discussion:

The implementation of the CodeGen-2B-Multi model demonstrates that Large Language Models can effectively bridge the gap between human intent and machine execution. By utilizing asynchronous FastAPI and half-precision quantization, we achieved a balance between high-level reasoning and real-time performance.

6. Conclusion:

"This project successfully established a unified AI ecosystem for automated code generation, debugging, and translation. By integrating a 2-billion parameter model into a lightweight web framework, we reduced the 'tool fragmentation' often faced by developers. The results prove that AI-driven synthesis significantly lowers the cognitive load of switching between programming languages.

References:

- Salesforce AI: CodeGen: An Open Large Language Model for Code with Multi-Step Program Synthesis (2022).
- Tiago, S., et al.: FastAPI: Modern, High-Performance Web Framework for Python (2019).
- Vaswani, A., et al.: Attention is All You Need: Transformer Architectures (2017).
- Brown, T., et al.: Language Models are Few-Shot Learners (2020).
- Documentation: Salesforce CodeGen-2B-Multi Model Card, Hugging Face (2023).